

Plan de Tests

Sujet : Développement d'un Assistant Pédagogique

Groupe L3S

Auteurs : Ines Yousfi
Mounir Lamouchi
Rayane Kachbi

Encadrant : David Janiszek

Version du document : 1.0

Confidentialité : Réservé aux étudiants UFR Maths-Info Paris Descartes.

Type de diffusion : Document électronique (.pdf)

Sommaire

1	Introduction	2
2	Guide de lecture	2
2.1	Maîtrise d'œuvre	2
2.2	Maîtrise d'ouvrage	2
3	Concepts de base	2
4	Tests fonctionnels	2
5	Tests d'intégration	3
5.1	Intégration du Module d'Ingestion	3
6	Tests Unitaires	4
6.1	Présentation des tests unitaires	4
6.2	Tests par module	4
	Module 1 : Ingestion	4
	Module 2 : RAG	5
	Module 3 : Filtrage	7
7	Tests Complémentaires	9
7.1	Tests de robustesse	9
7.2	Tests de performance	9
8	Vérification de la couverture des tests	9
8.1	Correspondance avec le cahier des charges	10
8.2	Cas supplémentaires identifiés	10
9	Annexes	10
	Annexe 01 : Catalogue des Jeux d'Essai	10
	Glossaire	12

1. Introduction

Ce document constitue le plan de tests du projet AP-UP, un assistant pédagogique basé sur l'architecture RAG (Retrieval-Augmented Generation). Il décrit l'ensemble des tests fonctionnels, d'intégration et unitaires mis en place pour valider le bon fonctionnement de l'application, en couvrant les trois modules principaux : Ingestion, RAG et Filtrage.

2. Guide de lecture

2.1. Maîtrise d'œuvre

- **Rayane Kachbi** : Module d'Ingestion
- **Ines Yousfi** : Module RAG et tests d'intégration
- **Mounir Lamouchi** : Module de Filtrage et sécurité

2.2. Maîtrise d'ouvrage

- **Encadrant** : David Janiszek — Université Paris Cité
- **Groupe** : L3S — Licence 3 MIAGE

3. Concepts de base

- **Chunk** : Fragment de texte issu du découpage d'un document pédagogique (500 tokens, 50 tokens de chevauchement).
- **Embedding** : Représentation vectorielle d'un texte en 768 dimensions produite par le modèle multilingual-e5-base.
- **RAG** : Architecture combinant recherche sémantique et génération de texte par un LLM.
- **SLM-as-a-judge** : Utilisation d'un petit modèle de langage (ministral-3 :3b) comme classifieur binaire (OUI/NON) pour valider la pertinence des questions et la fidélité des réponses.
- **Cosine similarity** : Mesure de similarité entre deux vecteurs utilisée pour la recherche sémantique et le filtrage.
- **Zone grise** : Plage de scores cosinus (0.30–0.50) pour laquelle le SLM est sollicité pour trancher.
- **Mock** : Objet simulé utilisé dans les tests unitaires pour isoler la logique métier des dépendances externes.
- **pytest** : Framework de tests Python utilisé pour les tests du module d'ingestion et du module de filtrage.
- **unittest** : Framework de tests Python intégré à la bibliothèque standard, utilisé pour les tests du module RAG.

4. Tests fonctionnels

Les tests fonctionnels ont pour objectif de vérifier que les fonctionnalités principales de l'application répondent correctement aux besoins des utilisateurs. Contrairement aux tests

unitaires, ils ne testent pas une fonction précise du code, mais un scénario complet réalisé depuis l'interface utilisateur.

Les principaux scénarios fonctionnels identifiés sont les suivants :

Connexion d'un utilisateur (étudiant et enseignant)

Un utilisateur saisit ses identifiants sur la page de connexion. Si les informations sont correctes, il accède à son espace de travail. Si le mot de passe est incorrect, l'application affiche un message d'erreur et refuse la connexion.

Upload d'un document de cours

Un enseignant importe un document PDF dans un module. Le document doit être accepté par l'application, traité par le pipeline d'ingestion, puis disponible pour les questions des étudiants.

Question posée par un étudiant

Un étudiant sélectionne un module et pose une question liée au cours. L'application doit générer une réponse contextualisée à partir des documents importés.

Question hors contexte

Un étudiant pose une question qui n'est pas liée au contenu du cours. L'application doit éviter d'inventer une réponse et indiquer que les documents disponibles ne permettent pas de répondre précisément.

Historique de conversation

Après plusieurs questions, l'étudiant doit pouvoir voir les échanges précédents dans l'interface de chat. L'historique doit conserver l'ordre des questions et des réponses.

5. Tests d'intégration

5.1. Intégration du Module d'Ingestion

L'objectif de ces tests est de vérifier que les bibliothèques d'extraction (PDFPlumber, Docling, Whisper, Unstructured) parviennent à traiter de vrais fichiers physiques.

Test d'intégration de la détection de type (`test_detect_type_integration.py`) :

- *Données d'entrée* : Fichiers réels de différents formats (`sample_simple.pdf`, `sample_columns.pdf`, `sample_scanned.pdf`, `sample_audio.mp3`).
- *Résultats attendus* : L'analyseur identifie correctement la complexité et renvoie le bon processeur ("simple", "complex", "scanned", "audio_video").
- *Critères d'évaluation* : Le type retourné par `detect_type()` correspond exactement au format réel du fichier.

Test d'intégration de l'extraction de texte (`test_extraction_integration.py`) :

- *Données d'entrée* : Fichiers PDF (simples, en colonnes, scannés) et fichiers audio.
- *Résultats attendus* : Le texte est correctement extrait pour chaque fichier. Docling doit lire les colonnes sans mélanger le texte, Whisper doit transcrire l'audio avec précision, et Unstructured doit appliquer l'OCR sur le PDF scanné.
- *Critères d'évaluation* : Présence de phrases clés spécifiques dans le texte extrait.

6. Tests Unitaires

6.1. Présentation des tests unitaires

Deux frameworks de tests ont été utilisés selon les modules :

pytest est utilisé pour le module d'Ingestion et le module de Filtrage. Il offre une syntaxe simple et des marqueurs permettant de séparer les tests rapides des tests d'intégration plus lourds (marqueur `@pytest.mark.slow`).

unittest est utilisé pour le module RAG. Il s'agit du framework de tests intégré à la bibliothèque standard Python, utilisé ici pour isoler la logique métier via des mocks.

Les commandes d'exécution par module sont les suivantes :

Module Ingestion (pytest) :

- Exécution globale : `pytest tests_ingestion/ -v`
- Logique pure uniquement : `pytest tests_ingestion/ -v -m "not slow"`
- Tests d'intégration uniquement : `pytest tests_ingestion/ -v -m "slow"`
- Mode débogage : `pytest tests_ingestion/ -x -v`

Module Filtrage (pytest) :

- Tous les tests filtrage : `pytest tests/test_filtrage/ -v`
- Validateurs uniquement : `pytest tests/test_filtrage/test_validators.py -v`
- Filtering pipeline : `pytest tests/test_filtrage/test_filtering.py -v`
- Test global : `pytest tests/test_filtrage/test_global.py -v`
- Mode débogage : `pytest tests/test_filtrage/ -x -v`

Module RAG (unittest) :

- Tous les tests RAG : `python -m pytest tests/test_rag/ -v`
- Retriever : `python -m pytest tests/test_rag/test_retriever.py -v`
- Prompt builder : `python -m pytest tests/test_rag/test_prompt_builder.py -v`
- RAG engine : `python -m pytest tests/test_rag/test_rag_engine.py -v`

6.2. Tests par module

Module 1 : Ingestion

Ce module est responsable de la préparation des documents avant leur vectorisation. Les tests unitaires valident le nettoyage du texte, la détection des structures mathématiques/-colonnes, et la logique complexe de découpage (chunking).

Sous-module : DocumentProcessor (Nettoyage et Routage)

Jeu d'essai 1 : Nettoyage du texte

- *Données d'entrée* : Chaînes contenant des espaces multiples, des retours à la ligne Windows (`\r\n`), ou des lignes vides consécutives.
- *Résultats attendus* : Le texte est normalisé (espaces réduits, sauts de ligne uniformisés, lignes vides supprimées).
- *Critères d'évaluation* : Égalité stricte entre la sortie de `clean_text()` et la chaîne attendue.

Jeu d'essai 2 : Détection des colonnes PDF

- *Données d'entrée* : Un faux objet mocké simulant un PDF avec des blocs de texte espacés horizontalement ($> 20\text{px}$), et une simulation de crash de PyMuPDF.
- *Résultats attendus* : `_est_en_colonnes()` retourne `True` pour le premier cas, et capture l'erreur sans faire planter l'application dans le second.
- *Critères d'évaluation* : La valeur booléenne est correcte et aucune exception n'est propagée.

Sous-module : chunking.py (Algorithme de découpage)*Jeu d'essai 3 : Découpage structurel (Markdown et Paragraphes)*

- *Données d'entrée* : Textes formatés avec des titres Markdown (`## Titre`) ou séparés par des doubles sauts de ligne.
- *Résultats attendus* : Le texte est divisé en segments distincts respectant la séparation logique.
- *Critères d'évaluation* : La taille de la liste résultante correspond au nombre de blocs logiques injectés.

Jeu d'essai 4 : Redécoupage forcé et comptage de tokens

- *Données d'entrée* : Un long paragraphe excédant la limite de `chunk_size` (simulé via un mock de `compter_tokens`).
- *Résultats attendus* : La fonction segmente la phrase en utilisant les sauts de ligne simples ou les points comme séparateurs de secours.
- *Critères d'évaluation* : Chaque sous-fragment respecte strictement la limite de tokens imposée.

Jeu d'essai 5 : Application du chevauchement (Overlap)

- *Données d'entrée* : Liste de trois chunks successifs avec un `chunk_overlap` paramétré à 3 tokens.
- *Résultats attendus* : Les derniers mots du chunk 1 se retrouvent au début du chunk 2.
- *Critères d'évaluation* : Présence de la chaîne `"overlap_"` injectée par le mock au début des chunks consécutifs.

Module 2 : RAG

Ce module génère une réponse contextualisée à partir des documents du cours. Il vectorise la question, récupère les chunks pertinents, construit un prompt enrichi, puis interroge le LLM en streaming. Les tests utilisent des mocks pour simuler les dépendances externes (modèle d'embedding, client LLM, base vectorielle).

Sous-module : Retriever*Jeu d'essai 1 : Vectorisation de la question*

- *Données d'entrée* : Une question textuelle, par exemple "Question de test".
- *Résultats attendus* : `vectoriser_question()` retourne le vecteur produit par le modèle d'embedding.
- *Critères d'évaluation* : La sortie est une liste non vide contenant uniquement des nombres flottants.

Jeu d'essai 2 : Recherche des chunks pertinents

- *Données d'entrée* : Une question, un `module_id` et une valeur `top_k`.

- *Résultats attendus* : `retrieve_chunks()` transmet le vecteur, le `module_id` et `top_k` à `rechercher_chunks()`.
- *Critères d'évaluation* : `rechercher_chunks()` est appelée avec les bons paramètres et retourne les chunks de la base mockée.

Jeu d'essai 3 : Aucun chunk trouvé

- *Données d'entrée* : Une recherche vectorielle qui retourne une liste vide.
- *Résultats attendus* : Le Retriever signale qu'aucun contexte n'est disponible.
- *Critères d'évaluation* : Une exception `ValueError` est levée avec un message contenant l'identifiant du module.

Sous-module : PromptBuilder

Jeu d'essai 4 : Construction du prompt final

- *Données d'entrée* : Une question, un prompt système et une liste de chunks.
- *Résultats attendus* : `build_prompt()` retourne une liste de messages avec le message système et le message utilisateur.
- *Critères d'évaluation* : Le premier message a le rôle "system", la question est présente dans le message utilisateur et l'en-tête "Contexte extrait du cours :" est inclus.

Jeu d'essai 5 : Formatage des chunks avec métadonnées

- *Données d'entrée* : Des chunks contenant du texte, une source, une section ou une page.
- *Résultats attendus* : Les chunks sont numérotés et affichent leurs métadonnées.
- *Critères d'évaluation* : Le prompt contient des références comme "[1] (source : cours.pdf, section : 1.1 Intro)".

Jeu d'essai 6 : Chunk sans source connue

- *Données d'entrée* : Un chunk contenant du texte mais aucune source.
- *Résultats attendus* : Le chunk reste utilisable dans le prompt.
- *Critères d'évaluation* : La source affichée est "inconnu".

Sous-module : RAGEngine

Jeu d'essai 7 : Pipeline RAG complet

- *Données d'entrée* : Une question, un `module_id`, des chunks et des messages simulés.
- *Résultats attendus* : `run()` appelle successivement `retrieve_chunks()`, `build_prompt()` et `stream_response()`.
- *Critères d'évaluation* : Les trois sous-composants sont appelés avec les bons paramètres et les tokens générés sont retournés dans l'ordre.

Jeu d'essai 8 : Streaming de la réponse

- *Données d'entrée* : Un flux LLM simulé contenant des tokens valides et des tokens vides.
- *Résultats attendus* : `stream_response()` retourne uniquement les tokens utiles.
- *Critères d'évaluation* : Les valeurs `None` et les chaînes vides sont ignorées et le client LLM est appelé avec `stream=True`.

Jeu d'essai 9 : Sauvegarde de l'échange

- *Données d'entrée* : La dernière question, la réponse générée, le `module_id` et les identifiants des chunks utilisés.
- *Résultats attendus* : `sauvegarder()` transmet ces informations à `sauvegarder_historique()`.

- *Critères d'évaluation* : `sauvegarder_historique()` est appelée avec `utilisateur_id`, `module_id`, `question`, `réponse` et `chunks_ids`.

Module 3 : Filtrage

Ce module est responsable de la sécurité et de la pertinence de l'application. Il valide les questions entrantes (filtrage entrée) et les réponses générées (filtrage sortie) via une logique hybride combinant similarité cosinus et SLM-as-a-judge (ministral-3 :3b). La suite de tests comprend 33 tests unitaires répartis sur trois fichiers.

Sous-module : FilterResult

Jeu d'essai 1 : Création d'un résultat valide

- *Données d'entrée* : `is_valid=True`, `reason="pertinent"`, `score=0.85`, `method="cosinus"`.
- *Résultats attendus* : L'objet `FilterResult` est créé avec les attributs corrects.
- *Critères d'évaluation* : `result.is_valid == True` et `result.score == 0.85`.

Jeu d'essai 2 : Création d'un résultat invalide

- *Données d'entrée* : `is_valid=False`, `reason="hors-sujet"`, `score=0.20`.
- *Résultats attendus* : L'objet représente un refus avec le score et la raison corrects.
- *Critères d'évaluation* : `result.is_valid == False` et `result.reason == "hors-sujet"`.

Sous-module : InputValidator

Jeu d'essai 3 : Question pertinente — SLM dit OUI

- *Données d'entrée* : Question liée au cours, SLM mocké retournant "OUI", score cosinus = 0.75.
- *Résultats attendus* : La question est acceptée.
- *Critères d'évaluation* : `result.is_valid == True` et `result.method == "slm"`.

Jeu d'essai 4 : Question hors-sujet — SLM dit NON

- *Données d'entrée* : Question sans rapport avec le cours, SLM mocké retournant "NON", score cosinus = 0.25.
- *Résultats attendus* : La question est bloquée.
- *Critères d'évaluation* : `result.is_valid == False`.

Jeu d'essai 5 : Cosine override (score > 0.84)

- *Données d'entrée* : Score cosinus = 0.87, SLM mocké retournant "NON".
- *Résultats attendus* : La question est acceptée malgré le refus du SLM.
- *Critères d'évaluation* : `result.is_valid == True` et `result.method == "cosine_override"`.

Jeu d'essai 6 : Fallback cosinus (SLM indisponible)

- *Données d'entrée* : SLM mocké levant une exception, score cosinus = 0.60.
- *Résultats attendus* : Le système bascule sur le fallback cosinus sans planter.
- *Critères d'évaluation* : `result.is_valid == True` et `result.method == "cosine"`.

Jeu d'essai 7 : Chunks vides — fallback cosinus

- *Données d'entrée* : Liste de chunks vide passée à l'`InputValidator`.
- *Résultats attendus* : Le système gère le cas sans erreur et retourne un résultat invalide.
- *Critères d'évaluation* : `result.is_valid == False` et aucune exception levée.

Sous-module : OutputValidator

Jeu d'essai 8 : Réponse fidèle — score >= 0.50

- *Données d'entrée* : Réponse générée, chunks sources, score cosinus = 0.65.

- *Résultats attendus* : La réponse est acceptée directement sans appel au SLM.
- *Critères d'évaluation* : `result.is_valid == True` et `result.method == "cosine"`.

Jeu d'essai 9 : Réponse bloquée — score < 0.30

- *Données d'entrée* : Réponse générée, chunks sources, score cosinus = 0.15.
- *Résultats attendus* : La réponse est bloquée directement sans appel au SLM.
- *Critères d'évaluation* : `result.is_valid == False` et `result.method == "cosine"`.

Jeu d'essai 10 : Zone grise — SLM dit OUI ($0.30 \leq \text{score} < 0.50$)

- *Données d'entrée* : Score cosinus = 0.40, SLM mocké retournant "OUI".
- *Résultats attendus* : La réponse est validée après passage par le SLM.
- *Critères d'évaluation* : `result.is_valid == True` et `result.method == "slm"`.

Jeu d'essai 11 : Zone grise — SLM dit NON ($0.30 \leq \text{score} < 0.50$)

- *Données d'entrée* : Score cosinus = 0.35, SLM mocké retournant "NON".
- *Résultats attendus* : La réponse est bloquée après passage par le SLM.
- *Critères d'évaluation* : `result.is_valid == False` et `result.method == "slm"`.

Sous-module : FilteringPipeline

Jeu d'essai 12 : Pipeline complet — question acceptée, réponse validée

- *Données d'entrée* : Question pertinente, SLM OUI en entrée et en sortie, scores cosinus élevés.
- *Résultats attendus* : La question et la réponse passent les deux filtres.
- *Critères d'évaluation* : Les deux `FilterResult` ont `is_valid == True`.

Jeu d'essai 13 : Pipeline — question bloquée, sortie non évaluée

- *Données d'entrée* : Question hors-sujet, SLM NON en entrée.
- *Résultats attendus* : La question est bloquée et le filtrage sortie n'est pas exécuté.
- *Critères d'évaluation* : `input_result.is_valid == False` et `output_result` est `None`.

Jeu d'essai 14 : Suggestions de questions alternatives

- *Données d'entrée* : Question bloquée, SLM mocké retournant 3 suggestions.
- *Résultats attendus* : Le pipeline retourne 3 questions alternatives extraites des chunks les plus proches.
- *Critères d'évaluation* : La liste de suggestions contient exactement 3 éléments non vides.

Jeu d'essai 15 : Déduplication des sources

- *Données d'entrée* : Liste de chunks avec des sources dupliquées (même PDF, même section).
- *Résultats attendus* : `format_references()` retourne une liste sans doublons.
- *Critères d'évaluation* : Chaque référence (PDF + section) n'apparaît qu'une seule fois dans la liste.

Test global d'intégration (test_global.py)

Ce fichier valide le comportement du pipeline de filtrage de bout en bout avec 11 questions couvrant trois catégories :

- **5 questions pertinentes** : Questions directement liées au contenu du cours. Résultat attendu : toutes acceptées.
- **3 questions hors-sujet** : Questions sans rapport avec le cours. Résultat attendu : toutes bloquées.

- **3 questions vagues** : Questions ambiguës nécessitant le double filtrage entrée + sortie. Résultat attendu : filtrage hybride cosinus + SLM appliqué.

7. Tests Complémentaires

7.1. Tests de robustesse

Au cours du développement, plusieurs scripts de validation ont été conçus pour éprouver la robustesse des pipelines et valider les choix technologiques de bout en bout.

Analyse structurelle et détection (Ingestion) : Des scripts ont permis d'explorer la structure brute des documents (comptage de mots, détection de la position spatiale des textes, extraction d'images) afin de calibrer les algorithmes de détection de complexité.

Persistance et cycle de vie des données (BDD) : Des cycles complets d'initialisation de la base et d'injection de données ont été simulés, validant la création en cascade des entités relationnelles (Enseignant → Module → Document) et l'insertion massive des chunks vectorisés via SQLAlchemy.

Connexion et streaming LLM (RAG) : Un script isolé a été utilisé pour valider la communication entre la base vectorielle et le modèle d'inférence, s'assurant que les requêtes complètes (avec contexte) étaient correctement transmises et que la réponse était bien restituée en flux continu (streaming).

Robustesse du filtrage (Filtrage) : Des tests exploratoires ont permis de comparer les comportements des modèles llama3.2, phi4-mini et ministral-3 :3b face à des instructions strictes OUI/NON, justifiant le choix final de ministral-3 :3b. Des cas limites ont également été testés manuellement : chunks vides, SLM hors ligne, scores à la frontière de la zone grise.

7.2. Tests de performance

Les tests de performance ont porté sur deux aspects principaux :

Temps de réponse du pipeline RAG : Le temps moyen de génération d'une réponse complète a été mesuré sur le serveur Pléiade avec le modèle phi4-mini. Le streaming permet d'afficher les premiers tokens rapidement, améliorant la perception de performance côté utilisateur.

Performance du filtrage : Le filtrage entrée (SLM + cosinus) ajoute une latence mesurable au traitement. Le cosine override (seuil 0.84) permet de court-circuiter l'appel au SLM pour les questions clairement pertinentes, réduisant ainsi le temps de réponse global. Le fallback cosinus garantit une disponibilité du filtrage même en cas d'indisponibilité du serveur Pléiade.

8. Vérification de la couverture des tests

Pour mesurer la couverture du code parcouru par les tests, nous utilisons l'extension **pytest-cov**. Cette vérification est appliquée spécifiquement au module d'ingestion via la

commande suivante :

```
pytest tests_ingestion/ --cov=src -v
```

Cette commande génère un rapport détaillant le pourcentage de lignes de code effectivement testées pour la brique d'extraction et de découpage du module d'ingestion.

8.1. Correspondance avec le cahier des charges

FP/FS	Fonctionnalité	Tests couvrants	Couvert
FP-04	Indexation automatique des documents	test_chunking.py, test_detect_type_integration.py	Oui
FP-05	Pose de question par un étudiant	test_validators.py (InputValidator)	Oui
FP-06	Recherche sémantique	test_retriever.py, test_rag_engine.py	Oui
FP-07	Génération de réponse	test_rag_engine.py, test_prompt_builder.py	Oui
FP-08	Filtrage de sécurité	test_validators.py, test_filtering.py, test_global.py	Oui
FS-05	Affichage des sources	test_filtering.py (format_references)	Oui

TABLE 1: Correspondance tests / cahier des charges

8.2. Cas supplémentaires identifiés

Au-delà des fonctionnalités prévues dans le cahier des charges, les tests couvrent plusieurs cas limites non prévus initialement :

- **SLM indisponible** : Test du fallback cosinus lorsque le serveur Pléiade est inaccessible.
- **Chunks sans section** : Test de la propagation automatique de section pour les chunks issus de tableaux ou listes sans titre Markdown.
- **Questions vagues** : Test du double filtrage (entrée + sortie) pour les questions ambiguës en zone grise.
- **Réponse hallucinée** : Test du blocage des réponses dont le score cosinus est inférieur à 0.30.
- **Suggestions alternatives** : Test de la génération de 3 questions alternatives lorsqu'une question est bloquée.

9. Annexes

Annexe 01 : Catalogue des Jeux d'Essai (Module Ingestion)

Un corpus de fichiers de tests standardisés a été créé et stocké dans le dossier `tests_ingestion/files/`. Ces fichiers couvrent les cas d'usage spécifiques du pipeline d'ingestion :

- `sample_simple.pdf` : Un document PDF purement textuel classique traité par PDFPlumber. Il contient une histoire narrative pour vérifier l'extraction linéaire et le découpage par paragraphes.
- `sample_columns.pdf` : Un document PDF en colonnes traité par Docling. Il sert à vérifier que l'extracteur ne mélange pas les mots de la colonne de gauche avec ceux de la colonne de droite.
- `sample_scanned.pdf` : Un PDF sans couche de texte sélectionnable. Il valide le déclenchement de la bibliothèque Unstructured et de l'OCR.
- `sample_audio.mp3` : Un fichier audio d'un extrait de cours. Il valide le déclenchement du modèle Whisper pour la transcription voix-texte.

Glossaire

- **Chunk** : Fragment de texte issu du découpage d'un document pédagogique.
- **Embedding** : Représentation numérique (vecteur) d'un texte encodant son sens sémantique.
- **LLM** : Large Language Model — modèle de langage de grande taille capable de générer du texte.
- **OCR** : Optical Character Recognition — reconnaissance optique de caractères pour les PDFs scannés.
- **pytest** : Framework de tests Python utilisé pour les modules d'ingestion et de filtrage.
- **RAG** : Retrieval-Augmented Generation — architecture combinant recherche documentaire et génération de texte.
- **SLM** : Small Language Model — modèle de langage léger utilisé comme juge binaire (OUI/NON).
- **Streaming** : Envoi de la réponse token par token au lieu d'attendre la réponse complète.
- **unittest** : Framework de tests intégré à Python, utilisé pour le module RAG.